

串与模式匹配

失配之后，怎样复用已经确认的前缀信息

408 · 数据结构 Topic DS-03

母问题：失配之后，哪些信息真的已经知道？

给定文本 T 和模式 P ，匹配不是“把两个字符串碰运气比较”，而是一个不断维护匹配状态的扫描过程：

Matched Prefix $\rightarrow$ Mismatch $\rightarrow$ Reusable Border

$\rightarrow$ Next Alignment $\rightarrow$ Continue / Accept.

朴素算法在失配后把模式移回下一个起点，因此反复比较文本；KMP 观察已匹配的模式前缀，寻找同时也是后缀的最长边界 (border)，只缩短模式状态。箭头表示一次匹配状态机的时间顺序，不是某个特定 'next' 数组下标的背诵顺序。

本册定位与 Owner 边界

- **Owns**：串的顺序对象、朴素匹配、prefix function/failure 信息、失配回退与 KMP 主循环。
- **Uses**：Atlas 的表示—操作—不变量—成本语言；底层字符序列只借用一般线性存储，不重新定义 DS01。
- **Outputs**：向算法专题提供线性时间模式匹配的可调用接口。
- **Stops**：完整 DFA/自动机最小化、多模式匹配与文本索引属于 Extension 或其他专题。

目录

1 对象、契约与第一基线	2
2 Prefix Function：模式自己的可复用结构	2
2.1 把 π 真正算一遍：ababaca	2
3 KMP 主循环：模式状态回退，主串不倒退	2
4 代码：把两种扫描策略放在同一契约下	3
4.1 朴素基线：每个起点重新建立 matched	3
4.2 Prefix function：失配时只沿 border 链回退	4
4.3 KMP：失配不倒退主串	4
4.4 测试：重复前后缀与边界返回值	5
5 复杂度与边界	6
6 概念边界与做题控制协议	7
7 全科坐标与来源处理	7

1 对象、契约与第一基线

串是一个有序字符序列；模式匹配的输出契约是：返回 P 在 T 中第一次出现的起始位置，若不存在则返回 ‘npos’。本册代码另约定空模式返回 0，这使边界行为显式而不依赖调用者猜测。

朴素匹配枚举起点 $s = 0, 1, \dots, |T| - |P|$ ，在每个起点从左到右比较。若在第 j 个字符失配，已经比较过的前缀信息只用于判定当前起点失败，算法不会把它带到下一起点。最坏比较次数可以达到 $\Theta((n-m+1)m)$ ，其中 $n = |T|, m = |P|$ 。

为什么需要新机制

文本 $T = \text{aaaaab}$ 、模式 $P = \text{aaab}$ 时，许多起点都会先匹配若干个 ‘a’ 再失配。朴素算法每次都重新确认这些 ‘a’；但 “已经看到的后缀由若干 ‘a’ 构成” 并没有消失。若模式自身存在重复前后缀，就可以把这份信息压缩成下一次应比较的模式位置。

2 Prefix Function: 模式自己的可复用结构

对模式前缀 $P[0..i]$ ，定义 $\pi[i]$ 为其最长真前缀且同时为后缀的长度。它只由模式自身决定，不依赖文本：

$$\pi[i] = \max\{k < i + 1 : P[0..k-1] = P[i-k+1..i]\}.$$

若当前边界长度为 $b = \pi[i-1]$ 且新字符不匹配，任何更短的候选边界都必然是长度 b 边界的边界，因此可令 $b \leftarrow \pi[b-1]$ ，继续尝试，而不用从 0 重新比较。

这给出构造不变量：处理 $P[0..i]$ 后，‘prefix[i]’ 恰是该前缀的最长 border；回退链严格变短，所以整个构造是线性的摊还过程。

2.1 把 π 真正算一遍：ababaca

逐个加入新字符，并始终问 “当前最长 border 能否延长；不能时，它自己的最长 border 是谁”：

i	$P[i]$	旧候选 b	比较、回退与可复用事实	$\pi[i]$
0	a	无	单字符没有非空真 border	0
1	b	0	$b \neq a$ ，无法延长	0
2	a	0	$a = a$ ，空 border 延长为 ‘a’	1
3	b	1	$b = P[1]$ ，‘a’ 延长为 ‘ab’	2
4	a	2	$a = P[2]$ ，‘ab’ 延长为 ‘aba’	3
5	c	3	$c \neq P[3]$ ：3 $\rightarrow$ 1 $\rightarrow$ 0，仍不等	0
6	a	0	$a = P[0]$ ，重新得到 ‘a’	1

因此 $\pi = [0, 0, 1, 2, 3, 0, 1]$ 。第 5 项按 3 $\rightarrow$ 1 $\rightarrow$ 0 回退，是因为长度 3 的 border 已失败，剩余候选只能是这个 border 自己的 border；其他长度不可能同时保持前缀和后缀。

3 KMP 主循环：模式状态回退，主串不倒退

扫描文本下标 i 时，变量 ‘matched’ 表示当前文本后缀与模式前缀相等的长度。失配时反复执行

$$\text{matched} \leftarrow \pi[\text{matched} - 1]$$

直到匹配、归零或接受完整模式；文本下标 i 仍只前进一次。若 `matched==m`，起点为 $i + 1 - m$ 。

一次完整失配：文本不回头，模式状态缩短

用 $P = \text{ababaca}$ 扫描文本片段 abababaca 。读完 ababa 后， $\text{matched} = 5$ ；下一文本字符是 `'b'`，而 $P[5] = \text{c}$ 。已知文本后缀 aba 等于模式前缀 aba ，所以令

$$5 \rightarrow \pi[4] = 3.$$

仍用同一个文本字符 `'b'` 比较 $P[3]$ ，匹配后变成 $\text{matched} = 4$ ；主串下标没有倒退，也没有重新比较已经证实的 aba 。这就是代码中 `'while'` 回退后紧接 `'if'` 比较的语义。

状态	迁移	必须保持的事实
$\text{matched}=0$	比较当前文本字符与 $P[0]$	没有可复用的模式前缀
$\text{matched}>0$ 且失配	沿 <code>prefix</code> 链缩短模式状态	不移动文本下标，保留已证实的后缀
匹配后	$\text{matched}++$	当前状态代表长度为 matched 的相等前缀
$\text{matched}=m$	返回 $i + 1 - m$	所有模式字符已连续匹配

4 代码：把两种扫描策略放在同一契约下

完整 C++17 实现位于 `code/ds03_string_matching.hpp`，测试位于 `code/ds03_string_matching_test.cpp`；正文代码块直接从真实源文件抽取。

4.1 朴素基线：每个起点重新建立 `matched`

Listing 1: 朴素匹配的起点枚举与局部比较

```
10 inline std::size_t naive_search(std::string_view text, std::string_view
    pattern) {
11     if (pattern.empty()) {
12         return 0;
13     }
14     if (pattern.size() > text.size()) {
15         return std::string_view::npos;
16     }
17     for (std::size_t start = 0; start + pattern.size() <= text.size(); ++
        start) {
18         std::size_t matched = 0;
19         while (matched < pattern.size() && text[start + matched] == pattern[
            matched]) {
20             ++matched;
21         }
22         if (matched == pattern.size()) {
23             return start;
24         }
25     }
26     return std::string_view::npos;
```

27 }

朴素版本的重要性在于提供复杂度基线：它没有错误，只是没有利用跨起点的结构信息。任何声称 KMP 更快的解释，都应能指出它省掉了哪类重复比较。

4.2 Prefix function：失配时只沿 border 链回退

Listing 2: prefix function 的回退链构造

```

30 std::vector<int> prefix(pattern.size(), 0);
31 for (std::size_t i = 1; i < pattern.size(); ++i) {
32     int border = prefix[i - 1];
33     while (border > 0 && pattern[i] != pattern[static_cast<std::size_t>(
border)]) {
34         border = prefix[static_cast<std::size_t>(border) - 1];
35     }
36     if (pattern[i] == pattern[static_cast<std::size_t>(border)]) {
37         ++border;
38     }
39     prefix[i] = border;
40 }
41 return prefix;
42 }
43
44 inline std::size_t kmp_search(std::string_view text, std::string_view
pattern) {
45     if (pattern.empty()) {
46         return 0;
47     }

```

‘border’ 只表示当前候选前缀长度。每次回退都跳到更短 border，因此不会形成无限循环；若新字符与候选前缀末字符相等，再增长一次。prefix 数组是 KMP 运行阶段可复用信息的唯一来源。

4.3 KMP：失配不倒退主串

Listing 3: KMP 主循环与完整模式接受

```

49 std::size_t matched = 0;
50 for (std::size_t i = 0; i < text.size(); ++i) {
51     while (matched > 0 && text[i] != pattern[matched]) {
52         matched = static_cast<std::size_t>(prefix[matched - 1]);
53     }
54     if (text[i] == pattern[matched]) {
55         ++matched;
56     }
57     if (matched == pattern.size()) {
58         return i + 1 - pattern.size();
59     }

```

```

60     }
61     return std::string_view::npos;
62 }
63
64 } // namespace ipara::ds03
65
66 #endif

```

主串循环变量 ‘i’ 单调递增；模式状态 ‘matched’ 才可能沿 prefix 链下降。于是预处理 $O(m)$ 加扫描 $O(n)$ ，总时间 $O(n + m)$ ，额外空间 $O(m)$ 。这不是 “next 公式自动带来线性”，而是由两个单调性共同保证。

4.4 测试：重复前后缀与边界返回值

Listing 4: 空模式、重复前后缀、失配回退与基线一致性

```

9  using ipara::ds03::naive_search;
10 using ipara::ds03::prefix_function;
11
12 void test_empty_and_boundary_patterns() {
13     assert(naive_search("abc", "") == 0);
14     assert(kmp_search("abc", "") == 0);
15     assert(naive_search("abc", "abcd") == std::string_view::npos);
16     assert(kmp_search("abc", "abcd") == std::string_view::npos);
17     assert(naive_search("", "a") == std::string_view::npos);
18     assert(kmp_search("", "a") == std::string_view::npos);
19 }
20
21 void test_naive_and_kmp_agree() {
22     const std::vector<std::pair<std::string_view, std::string_view>> cases
23         = {
24         {"abccabcbcd", "abccabcbcd"}, {"aaaaaab", "aaab"}, {"mississippi", "issi"},
25         {"abcdef", "gh"}, {"abababab", "ababa"};
26     for (const auto& [text, pattern] : cases) {
27         assert(naive_search(text, pattern) == kmp_search(text, pattern));
28     }
29 }
30
31 void test_prefix_reuse_information() {
32     assert((prefix_function("ababaca") == std::vector<int>{0, 0, 1, 2, 3, 0, 1}));
33     assert((prefix_function("aaaa") == std::vector<int>{0, 1, 2, 3}));
34     assert((prefix_function("aabaaab") == std::vector<int>{0, 1, 0, 1, 2, 2, 3}));
35 }
36
37 void test_repeated_mismatch_does_not_lose_text_progress() {

```

```
37  assert(kmp_search("ababababca", "abababca") == 2);
38  assert(kmp_search("aaaaaaaaab", "aaab") == 6);
39  }
40
41  std::vector<std::string> binary_strings(std::size_t maximum_length) {
42      std::vector<std::string> strings{" "};
43      for (std::size_t length = 1; length <= maximum_length; ++length) {
44          const std::size_t count = std::size_t{1} << length;
45          for (std::size_t mask = 0; mask < count; ++mask) {
```

测试要求朴素与 KMP 在多组文本/模式上返回同一首个位置，直接检查 `prefix` 数组的重复结构，并覆盖空模式、模式过长、完全无匹配和失配后仍有答案的情形。

测试还枚举二字母表上长度不超过 6 的全部文本、长度不超过 4 的全部模式，逐对验证 KMP 与朴素算法返回同一首个位置。穷举不是正确性证明，但会系统攻击空模式、重叠匹配、周期串、长 `border` 链与无匹配状态。

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \\
code/ds03_string_matching_test.cpp -o /tmp/ds03_test
/tmp/ds03_test
```

5 复杂度与边界

任务	朴素匹配	KMP	关键前提	失败边界
预处理模式	无	$O(m)$	<code>prefix</code> 语义固定	空模式
扫描文本	$O((n - m + 1)m)$ 最坏	$O(n)$	主串下标单调前进	$m > n$
总额外空间	$O(1)$	$O(m)$	只保存模式信息	大模式内存预算
返回位置	首个合法起点	首个合法起点	接受后立即返回	无匹配返回 <code>npos</code>

不同教材的 ‘`next`’ 可能使用 “失配时跳到的下标” 或 “最长 `border` 长度” 两种编码。做题时先写语义，再把题目下标转换到该语义；不能拿另一版本的初值和边界直接替换。

从长度语义翻译到教材 `next`

本册只维护 $\pi[i] = \text{“}P[0..i] \text{的最长 } border \text{ 长度”}$ 。若教材把 ‘`next[j]`’ 定义为 “模式状态 j 失配后跳到的状态”，零基长度状态下对应 $\pi[j - 1]$ ($j > 0$)。若教材使用 `-1` 哨兵或一基下标，先画出 “已经匹配多少个字符”，再平移下标；数组首项长什么样不是算法本体。

6 概念边界与做题控制协议

概念 A	≠ 概念 B	真正区别	混淆后果
串	≠ 模式	被搜索的文本对象 vs 约束模板	方向写反
border	≠ 任意重复片段	必须同时是前缀和后缀	错误回退位置
prefix function	≠ 匹配答案	模式内部信息 vs 文本中的出现位置	把预处理当搜索
失配回退	≠ 主串回退	只缩短模式状态	复杂度退化
线性时间	≠ 每次比较 O(1)	总比较次数还需证明单调性	复杂度口号化

遇到匹配题，按“固定返回契约 → 写朴素基线 → 找重复前后缀 → 固定 prefix/next 语义 → 画一次失配回退 → 再报告复杂度”的顺序处理。忘掉公式后，只需复原：**已经匹配了什么？失配时哪些后缀仍可能成为模式前缀？模式状态能否缩短而不重扫文本？**

7 全科坐标与来源处理

Map Coordinate: Prefix / Failure / Reuse

DS03 把线性序列上的“匹配失败”转化为模式内部 border 结构；它接收 DS01 的顺序访问成本语言，并向算法专题输出 KMP 的线性扫描接口。自动机、多模式和文本索引是后续 Extension，不改变本册唯一 Owner。

本册吸收归档总册对 KMP 的核心表述“已匹配前缀压缩为失败状态”，并将 next 下标差异明确为表示约定问题。代码和测试只验证本册固定的 prefix-length 语义，避免把不同教材的数组编码混作第二套稳定真相。